

Java Mobile Number Validation

Custom exception handling and input validation

Objective / Aim

To write a Java program that checks whether a mobile number contains digits only and has exactly 10 characters, demonstrating custom exception handling and built-in exceptions.

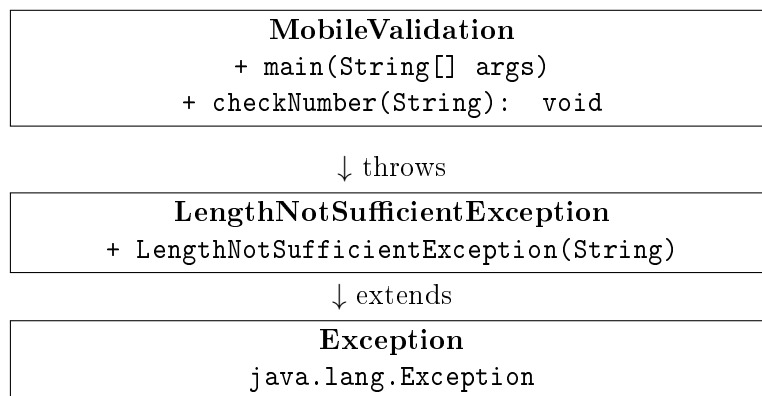
System Requirements

- **Hardware:** Standard computer with a Core i3/i5-class processor and 4 GB or more RAM.
- **Software:** Windows, Linux, or macOS; JDK 17 or JDK 21; and a text editor or Java IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans.

Algorithm / Logic Description

1. Read a mobile number as a string so that every character can be checked.
2. Scan each character. If a character is not a digit, throw and catch `NumberFormatException`.
3. If the number has more than 10 characters, copy the characters into a 10-element array. An input longer than 10 digits exceeds the array bounds and is caught as `ArrayIndexOutOfBoundsException`.
4. If the number has fewer than 10 characters, throw the custom `LengthNotSufficientException`; the caller catches it and prints its message.
5. Otherwise, print that the number is valid. The `finally` block prints a completion message and closes the scanner for every input case.

UML Class Diagram



`MobileValidation` throws the custom exception when the number is too short; `LengthNotSufficientException` inherits from Java's `Exception` class.

Source Code

Save the following program as `MobileValidation.java`:

```

import java.util.Scanner;

class LengthNotSufficientException extends Exception {

    LengthNotSufficientException(String message) {
        super(message);
    }
}

public class MobileValidation {

    static void checkNumber(String number)
        throws LengthNotSufficientException {

        try {

            // Check characters
            for (int i = 0; i < number.length(); i++) {

                if (!Character.isDigit(number.charAt(i))) {
                    throw new NumberFormatException();
                }
            }

            // More than 10 digits
            if (number.length() > 10) {

                int[] digits = new int[10];

                for (int i = 0; i < number.length(); i++) {
                    digits[i] = number.charAt(i) - '0';
                }

                // Less than 10 digits
                if (number.length() < 10) {
                    throw new LengthNotSufficientException(
                        "Invalid Mobile Number - LengthNotSufficientException"
                    );
                }

                System.out.println("Valid number");
            } catch (ArrayIndexOutOfBoundsException e) {

                System.out.println(
                    "Invalid Mobile Number-ArrayIndexOutOfBoundsException");
            } catch (NumberFormatException e) {

                System.out.println(
                    "Invalid Mobile Number -NumberFormatException");
            }
        }

    }

    public static void main(String[] args) {

```

```

Scanner sc = new Scanner(System.in);

System.out.print("Enter mobile number: ");
String number = sc.nextLine();

try {
    checkNumber(number);
}
catch (LengthNotSufficientException e) {
    System.out.println(e.getMessage());
}
finally {
    System.out.println("Checking completed.");
    sc.close();
}
}
}

```

Compilation & Execution Commands

Open a terminal in the folder containing `MobileValidation.java`, then run:

```

javac MobileValidation.java
java MobileValidation

```

Output Screen

The exact output depends on the entered value. Example runs:

Exactly 10 digits

```

Enter mobile number: 9876543210
Valid number
Checking completed.

```

Fewer than 10 digits

```

Enter mobile number: 987654321
Invalid Mobile Number - LengthNotSufficientException
Checking completed.

```

More than 10 digits

```

Enter mobile number: 98765432101
Invalid Mobile Number-ArrayIndexOutOfBoundsException
Checking completed.

```

Contains a nondigit

```

Enter mobile number: 98765abcde
Invalid Mobile Number -NumberFormatException
Checking completed.

```